

Short Vector Processing

1.3.7

SCANLAB GmbH

Thu May 25 2023 10:47:36

Contents

1	Short vector handling	1
2	Module Index	3
2.1	Modules	3
3	Module Documentation	4
3.1	Short vector optimization	4
3.1.1	Detailed Description	5
3.1.2	Function Documentation	5
3.2	Identification Routines	14
3.2.1	Detailed Description	14
3.2.2	Function Documentation	14
3.3	Multi-Board short vector functions	15
3.3.1	Detailed Description	15
3.3.2	Function Documentation	16
3.4	Multi-Board identification routines	23
3.4.1	Detailed Description	23
3.4.2	Function Documentation	23
4	Example Documentation	24
4.1	Example.cpp	24
	Index	26

1 Short vector handling

Package for accelerated marking of short vectors

The algorithm provided in this package handles the optimized execution of short marks. Especially collinear short vectors with small gaps in between are executed in shorter time. The algorithm shifts the vectors to reduce the need for long laser delays. This avoids the problem of overlapping laser off and on counters.

System Requirements

- IBM-compatible PC
- RTCx PC interface board
- Operating system: Microsoft WINDOWS 10, WINDOWS 8, WINDOWS 7.

Software Installation

Copy the supplied files onto your PC's hard disk: Copy all RTCxAddOn files into the same directory as your RTCx files.

Functionality

- An arbitrary number of mark and jump commands can be handed to a preprocessor. This class executes a check for collinearity and then adds the mark or jump commands to a command buffer.
- The command buffer holds all necessary RTC list commands to execute the previously preprocessed mark and jump sequence. The user can send the commands to an RTC list at any time.
- All commands will be inserted using standard RTC list commands. The list to hold and execute these commands must be handled (started, finalized and executed) by the application program!

Please refer to [Short vector optimization](#) for details on usage of the short vectors library.

Note

- Short Vector Library functions have to be unlocked by password (see [activateShortVectors](#)).
- Using Short Vector Library functions requires minimum RTCx-File-Version (see RTCxAddOnDLL_XXX_-VersionInfo.txt). In case of version mismatch [initShortVectors](#) returns an error.
- Laser synchronization will not function properly in combination with the short vector handling!
- Using Short Vector functions in combination with FLY is not recommended.
- In case of using RTC6 combined with a scan head with SCANAhead control it is mandatory to activate the RTC6-SCANAhead functionality via `set_scanahead_params` at the latest before starting the list execution.
- As the scanner's time lag and transport delay are required to use the Short Vector Library, identification functions giving initial estimates are provided (see [Identification Routines](#)).
- The mark speed must be set during initialization - the jump speed set in the application program will not be affected!
- To ensure proper functionality all scanner delays need to be zero - call `set_scanner_delays(0, 0, 0)` within your RTC-List! For RTC6 in addition all laser delays need to be zero - call `set_laser_delays(0, 0)` within your RTC-List.
- If a scan head with SCANAhead functionality is used, scanahead auto delays are automatically deactivated by Short-Vector routine - calling `activate_scanahead_autodelays_list(0)` In Addition the scanahead laser shifts are changed by the Short-Vector routine within the call of [sendShortVectorCommandsToRtc\(\)](#)
- The short vector handling will ignore a series of consecutive jumps! Only the last jump command before the following mark is considered!
- Before sending commands from the buffer to the RTC, the user must have started an RTC list!

Changes

Date

October 2014

- Added multi-board functions

November 2015

- Updated documentation

May 2016

- Interface changed for time-lag and extra pre run time. Both are now double values!
- New feature: Pre trigger time to account for delayed laser switching (see [setLaserPreTriggerTime](#))

August 2016

- Advanced handling of consecutive mark commands
- Configurable minimum vector length (see [setMinVectorLengthBits](#))

March 2017

- Fixed bug in 3D velocity calculation

May 2017

- Advanced handling for sequences (polylines) of consecutive mark commands

August 2017

- Fixed bug of import declarations - using PTR3 instead of PTR

March 2018

- Advanced processing of various consecutive short collinear marks of duration less than 10 us separated by jumps of duration greater than 10 us

April 2018

- Fixed bug for polylines - processing chains of collinear marks

August 2018

- Software package update including RTC6AddOn

December 2018

- New feature: scan heads with SCANahead control (excelliSCAN) are supported (see [initShortVectorsSCANa](#))
- Fixed bug for polylines - processing chains of non collinear marks
- Advanced functionality of [finalizeShortVectorInput](#)

May 2019

- New feature: configurable laser signal shift (see [setLaserSignalShiftSCANa](#)) for scan heads with SCANahead control (excelliSCAN)

November 2019

- RTC6 only: Advanced handling for dashed lines of very short marks and jumps. The minimum time between marks or the minimum permissible jump duration has been reduced from a minimum value greater than 10 us to a minimum value greater than 1us.
- New feature: Sets a minimum jump length (see [setMinJumpLengthBits](#))

December 2020

- New feature: Reads back the required rtc6 list memory space of the current handler (see [getNeededListSpace](#))
- New feature: Sets a limit value from which a warning is issued because the number of list entries in the current handler has been exceeded (see [setExecutionListSizeLimit](#))

January 2021

- Fixed bug for function [setLaserSignalShiftSCANa](#)

March 2021

- New feature: Sets a wait time before start of pre-run (see [setPreRunWaitTime](#))

October 2021

- Optimization of command transfer time of function [sendShortVectorCommandsToRtc](#)

May 2023

- Fixed bug for short jumps - missing parts of the following mark depending on timing situation

2 Module Index

2.1 Modules

Here is a list of all modules:

Short vector optimization

4

Identification Routines	14
Multi-Board short vector functions	15
Multi-Board identification routines	23

3 Module Documentation

3.1 Short vector optimization

Interface functions to use the short vector handling algorithms.

Macros

- #define [JUMP_COMMAND](#) 0
Identifier for jumps in [addShortVectorCommand](#).
- #define [MARK_COMMAND](#) 1
Identifier for marks in [addShortVectorCommand](#).
- #define [SHORT_MARK_COMMAND](#) 2
Identifier for short marks - special laser switching mode only! - in [addShortVectorCommand](#).

Functions

- RTCADDONDLL_API LONG [activateShortVectors](#) (const PTR3 Ptr)
Activates short vector processing by password.
- RTCADDONDLL_API LONG [initShortVectors](#) (LONG *Handler, double TimeLag, UINT TransportDelay, double Speed)
Initializes a Short Vector Handler with dynamic parameters and desired velocity.
- RTCADDONDLL_API LONG [initShortVectorsSCANa](#) (LONG *Handler, UINT PreviewTime, UINT Vmax, double Amax, double Speed)
Initializes a Short Vector Handler with dynamic parameters and desired velocity for a scan system with SCANahead control.
- RTCADDONDLL_API LONG [setExtraPreRunTime](#) (LONG Handler, double ExtraPreRunTime)
Sets an additional TimeLag value to increase acceleration distance for first vector of a collinear vector sequence.
- RTCADDONDLL_API LONG [setLaserPreTriggerTime](#) (LONG Handler, double LaserPreTriggerTime)
Sets a pre trigger time for lasers to account for delayed switching.
- RTCADDONDLL_API LONG [setLaserSignalShiftSCANa](#) (LONG Handler, double LaserSignalShift)
Sets a time shift for laser signal switching.
- RTCADDONDLL_API LONG [setPreRunWaitTime](#) (LONG Handler, double PreRunWaitTime)
Sets an additional wait time before each start of a pre-run-sequence.
- RTCADDONDLL_API LONG [setMinVectorLengthBits](#) (LONG Handler, UINT MinVectorLengthBits)
Sets a minimum vector length.
- RTCADDONDLL_API LONG [setMinJumpLengthBits](#) (LONG Handler, UINT MinJumpLengthBits)
Sets a minimum jump length.
- RTCADDONDLL_API LONG [addShortVectorCommand](#) (LONG Handler, double xPos, double yPos, LONG Type)
Adds a new vector command to the preprocessor.
- RTCADDONDLL_API LONG [addShortVectorCommand3D](#) (LONG Handler, double xPos, double yPos, double zPos, LONG Type)
Adds a new 3D vector command to the preprocessor.
- RTCADDONDLL_API LONG [finalizeShortVectorInput](#) (LONG Handler)

As the algorithm has an internal memory to recognize patterns of collinear vectors there is the possible need to keep the last command internally before adding it to the execution queue.

Calling this function sends a possible last command kept by the handler to the execution queue.

To finalize the sequence the add off an appropriate post-run is included.

- RTCADDONDLL_API LONG [setExecutionListSizeLimit](#) (LONG Handler, LONG ExeListLimit)

Sets a warning limit for stored execution list size (as number of entrys) to avoid a potential memory exception caused to memory allocation failed.

(This limit value can also be used as a control value for the number of execution list entrys. To estimate the list space requirement on the RTC this value can be approximatly doubled.)

- RTCADDONDLL_API LONG [getNeededListSpace](#) (LONG Handler)

Reads back the needed list space value of the actual handler.

- RTCADDONDLL_API LONG [sendShortVectorCommandsToRtc](#) (LONG Handler)

Transfers all commands in the command buffer to the RTCx list handled by the application program.

- RTCADDONDLL_API LONG [resetShortVectors](#) (LONG Handler)

Resets the internal state of a specific short vector object.

- RTCADDONDLL_API LONG [freeShortVectors](#) (LONG Handler)

Releases a specific short vector object.

- RTCADDONDLL_API LONG [freeAllShortVectors](#) ()

Releases ALL handlers for short vector calculation.

3.1.1 Detailed Description

Interface functions to use the short vector handling algorithms.

- Initialize the short vector handler by calling [initShortVectors](#) In case of using RTC6 combined with a scan head with SCANahead control use [initShortVectorsSCANa](#)
- Commands can be added any time and in any number using [addShortVectorCommand](#) and will then be kept in an internal command buffer.
- To insert the commands into an existing RTC list call [sendShortVectorCommandsToRtc](#)

Note

Due to the internal handling of collinear vectors there might be a last mark command in the internal storage waiting for execution. To make sure this command is transferred to the execution list use [finalizeShortVector-Input!](#)

To interface with the internal functionality, you need a reference pointer - called handler. The handler is generated during the initialization and needs to be handed over with every interface function call. One can construct as many handlers as needed and selectively free or reset them using [freeShortVectors](#) or [resetShortVectors](#). To free all handlers and clean up memory call [freeAllShortVectors](#).

3.1.2 Function Documentation

3.1.2.1 RTCADDONDLL_API LONG activateShortVectors (const PTR3 Ptr)

Activates short vector processing by password.

Returns

- 0 for success - activation code is valid
- 1 activation code is invalid

Examples:

[Example.cpp](#).

3.1.2.2 RTCADDONDLL_API LONG addShortVectorCommand (LONG *Handler*, double *xPos*, double *yPos*, LONG *Type*)

Adds a new vector command to the preprocessor.

Note

For the allowed range of xyz coordinates see the corresponding commands (mark_abs, jump_abs) of the RTC-Manual.

Parameters

in	<i>Handler</i>	Short vector handler address
in	<i>xPos</i>	x coordinate [Bits]
in	<i>yPos</i>	y coordinate [Bits]
in	<i>Type</i>	For JUMP 0, for MARK 1

Returns

- 0 for success
- 1 object handler is invalid
- 2 memory exception
- 3 command type is invalid
- 4 vector length is too short
- 5 warning: execution list size limit exceeded (further description see setExecutionListSizeLimit)

Examples:

[Example.cpp](#).

3.1.2.3 RTCADDONDLL_API LONG addShortVectorCommand3D (LONG *Handler*, double *xPos*, double *yPos*, double *zPos*, LONG *Type*)

Adds a new 3D vector command to the preprocessor.

Note

For the allowed range of xyz coordinates see the corresponding commands (mark_abs_3d, jump_abs_3d) of the RTC-Manual.

Parameters

in	<i>Handler</i>	Short vector handler address
in	<i>xPos</i>	x coordinate [Bits]
in	<i>yPos</i>	y coordinate [Bits]
in	<i>zPos</i>	z coordinate [Bits]
in	<i>Type</i>	For JUMP 0, for MARK 1

Returns

- 0 for success
- 1 object handler is invalid
- 2 memory exception
- 3 command type is invalid
- 4 vector length is too short
- 5 warning: execution list size limit exceeded (further description see setExecutionListSizeLimit)

3.1.2.4 RTCADDONDLL_API LONG finalizeShortVectorInput (LONG Handler)

As the algorithm has an internal memory to recognize patterns of collinear vectors there is the possible need to keep the last command internally before adding it to the execution queue.

Calling this function sends a possible last command kept by the handler to the execution queue.

To finalize the sequence the add off an appropriate post-run is included.

Parameters

in	Handler	Short vector handler
----	---------	----------------------

Returns

- 0 for success
- 1 object handler is invalid

Examples:

[Example.cpp](#).

3.1.2.5 RTCADDONDLL_API LONG freeAllShortVectors ()

Releases ALL handlers for short vector calculation.

Returns

- 0 for success
- 1 in case of an error

3.1.2.6 RTCADDONDLL_API LONG freeShortVectors (LONG Handler)

Releases a specific short vector object.

Parameters

in	Handler	Short vector handler to be released
----	---------	-------------------------------------

Returns

- 0 for success
- 1 object handler is invalid

Examples:

[Example.cpp](#).

3.1.2.7 RTCADDONDLL_API LONG getNeededListSpace (LONG Handler)

Reads back the needed list space value of the actual handler.

Parameters

in	Handler	Short vector handler address
----	---------	------------------------------

Returns

- 0 in case of execution list is empty
- ListSpace in case of execution list is not empty
- 1 object handler invalid
- 2 memory exception

3.1.2.8 RTCADDONDLL_API LONG initShortVectors (LONG * Handler, double TimeLag, UINT TransportDelay, double Speed)

Initializes a Short Vector Handler with dynamic parameters and desired velocity.

The function will construct a new short vectors object and returns a handler to interface with it. If the handler parameter points to an existing object, this object will be destroyed and the output value of the handler will point to a new object with the desired parameters.

Parameters

out	<i>Handler</i>	Memory Address of the newly constructed short vector handler - use this to reference the object in every function call!
in	<i>TimeLag</i>	Controller time lag (depends on scan head tuning) [us]
in	<i>TransportDelay</i>	Time needed to transfer RTCx commands to the scan head [us]
in	<i>Speed</i>	[Bits/ms]

Note

Time lag will be checked to be at least 20 us!

Returns

- 0 for success
- 1 if memory allocation failed
- 2 for bad input
- 3 ShortVectors functions access denied
- 4 version mismatch of RTCxAddOnDLL version and RTCx DLL, HEX (OUT-File) or RTC (RBF-File) version

Examples:

[Example.cpp](#).

3.1.2.9 RTCADDONDLL_API LONG initShortVectorsSCANa (LONG * Handler, UINT PreviewTime, UINT Vmax, double Amax, double Speed)

Initializes a Short Vector Handler with dynamic parameters and desired velocity for a scan system with SCANahead control.

The function will construct a new short vectors object and returns a handler to interface with it. If the handler parameter points to an existing object, this object will be destroyed and the output value of the handler will point to a new object with the desired parameters.

Note

This function only works with RTC6 and SCANa option enabled. In addition it is mandatory to activate the RTC6-SCANahead functionality via set_scanahead_params at the latest before starting the list execution.

Parameters

out	<i>Handler</i>	Memory address of the newly constructed short vector handler - use this to reference the object in every function call!
in	<i>PreviewTime</i>	Pre-calculation time for galvanometer scanner control in [10 us]
in	<i>Vmax</i>	Velocity limit as control bits (galvanometer scanner rotation angle) [bits / 10 us]
in	<i>Amax</i>	Acceleration limit as control bits (galvanometer scanner rotation angle) [bits / 10 us]
Generated on Thu May 25 2023 10:47:36 for ShortVector Processing by Doxygen		
in	<i>Speed</i>	[Bits/ms]

Returns

- 0 for success
- 1 if memory allocation failed
- 2 for bad input
- 3 ShortVectors functions access denied
- 4 version mismatch of RTCxAddOnDLL version and RTCx DLL, HEX (OUT-File) or RTC (RBF-File) version
- 6 Speed value exceeds Vmax

Examples:

[Example.cpp](#).

3.1.2.10 RTCADDONDLL_API LONG resetShortVectors (LONG Handler)

Resets the internal state of a specific short vector object.

The command buffer and a possible last command held internally are cleared.

Parameters

in	Handler	Short vector handler to be reset
----	---------	----------------------------------

Returns

- 0 for success
- 1 object handler is invalid

3.1.2.11 RTCADDONDLL_API LONG sendShortVectorCommandsToRtc (LONG Handler)

Transfers all commands in the command buffer to the RTCx list handled by the application program.

Note

This function may only be called from an application which has initialized the RTCxDLL and started an RTC list before calling!!

Parameters

in	Handler	Short vector handler
----	---------	----------------------

Returns

- Number of list commands written to the RTC
- 1 in case of a failure
- 2 in case list memory is full

Examples:

[Example.cpp](#).

3.1.2.12 RTCADDONDLL_API LONG setExecutionListSizeLimit (LONG Handler, LONG ExeListLimit)

Sets a warning limit for stored execution list size (as number of entries) to avoid a potential memory exception caused to memory allocation failed.

(This limit value can also be used as a control value for the number of execution list entries. To estimate the list space requirement on the RTC this value can be approximatly doubled.)

Note

- If the limit is exceeded addShortVectorCommand returns with ReturnCode = -5.
This return code is not to be understood as an error code, but only as a warning and has no effect on the execution of the addShortVectorCommand function.
It advises you to call the sendShortVectorCommandsToRtc command as soon as possible to empty the handler's cached execution list to free memory.
- One entry of the execution list needs about 440 byte RAM.
Recommended value for parameter ExeListLimit:
 $\text{ExeListLimit} = \text{available memory [byte]} / 440 \text{ byte}$
- The limit check is deactivated by default initialization.
It needs to be explicitly activated by calling the function with parameter $\text{ExeListLimit} > 0$.
For deactivation call the function with parameter $\text{ExeListLimit} \leq 0$.

Parameters

in	<i>Handler</i>	Short vector handler address
in	<i>ExeListLimit</i>	warning limit for handler execution list size

Returns

- 0 in case of success
- 1 object handler invalid
- 2 memory exception

3.1.2.13 RTCADDONDLL_API LONG setExtraPreRunTime (LONG Handler, double ExtraPreRunTime)

Sets an additional TimeLag value to increase acceleration distance for first vector of a collinear vector sequence.

Note

If the first vector / vectors of a collinear dash line are to short or to long the value of ExtraPreRunTime needs to be increased.

Parameters

in	<i>Handler</i>	Short vector handler address
in	<i>ExtraPreRunTime</i>	TimeLag value to extend acceleration distance [us]

Returns

- 0 for success
- 1 object handler or command type is invalid
- 2 memory exception

Examples:

[Example.cpp](#).

3.1.2.14 RTCADDONDLL_API LONG setLaserPreTriggerTime (LONG Handler, double LaserPreTriggerTime)

Sets a pre trigger time for lasers to account for delayed switching.

The laser on signal will be set as specified before the marking vector starts.

Parameters

in	<i>Handler</i>	Short vector handler address
in	<i>LaserPreTriggerTime</i>	Time to compensate for delayed switching (≥ 0)

Returns

- 0 for success
- 1 object handler or command type is invalid
- 2 memory exception

Examples:

[Example.cpp](#).

3.1.2.15 RTCADDONDLL_API LONG setLaserSignalShiftSCANa (LONG *Handler*, double *LaserSignalShift*)

Sets a time shift for laser signal switching.

The laser signals will be shifted by the specified time shift.

Note

This function only works with RTC6 and SCANa option enabled.
If a time shift shall be executed, it needs to be called once before the first call of `sendShortVectorCommandsToRtc`.
Please remark, the value must not be changed within a RTC list.

Parameters

in	<i>Handler</i>	Short vector handler address
in	<i>LaserSignalShift</i>	Time shift [us] to synchronize laser switching and scanner movement (allowed range -20us to +20us)

Returns

- 0 for success
- 1 object handler or command type is invalid
- 2 memory exception
- 3 for *LaserSignalShift* value is out of range and has been clipped
- 4 for SCANa option is not enabled

Examples:

[Example.cpp](#).

3.1.2.16 RTCADDONDLL_API LONG setMinJumpLengthBits (LONG *Handler*, UINT *MinJumpLengthBits*)

Sets a minimum jump length.

Only jump vectors of at least the specified minimum length will be added to the preprocessor.

Note

The minimum length must be at least equivalent to a jump duration of 0.5 us (RTC6) / 10 us (RTC5). Shorter jumps cannot be processed properly.

Parameters

in	<i>Handler</i>	Short vector handler address
in	<i>MinJumpLength-Bits</i>	Minimum jump length in Bits

Returns

- 0 for success
- 1 object handler invalid or MinJumpLengthBits too small
- 2 memory exception

3.1.2.17 RTCADDONDLL_API LONG setMinVectorLengthBits (LONG Handler, UINT MinVectorLengthBits)

Sets a minimum vector length.

Only mark vectors of at least the specified minimum length will be added to the preprocessor.

Note

The minimum length must be at least equivalent to a vector duration of 0.5 us. Shorter vectors cannot be processed properly.

Parameters

in	<i>Handler</i>	Short vector handler address
in	<i>MinVectorLengthBits</i>	Minimum vector length in Bits

Returns

- 0 for success
- 1 object handler invalid or MinVectorLengthBits too small
- 2 memory exception

3.1.2.18 RTCADDONDLL_API LONG setPreRunWaitTime (LONG Handler, double PreRunWaitTime)

Sets an additional wait time before each start of a pre-run-sequence.

Note

It is allowed to change the value of parameter PreRunWaitTime while adding ShortVectorCommands to the ShortVectorHandler.

The set value for PreRunWaitTime applies to the current call of addShortVectorCommand and all subsequent calls until the value is changed by calling [setPreRunWaitTime\(\)](#) with another parameter value.

By default initialization PreRunWaitTime is set to 0.

Parameters

in	<i>Handler</i>	Short vector handler address
in	<i>PreRunWaitTime</i>	Wait time [us] between post-run und pre-run of two non-collinear vectors (allowed range 0 us to 1,000,000 us) The parameter is rounded to an integer-multiple of 10

Returns

- 0 for success
- 1 object handler or command type is invalid

- 2 memory exception
- 3 for PreRunWaitTime value is out of range and has been clipped

3.2 Identification Routines

Helper functions to identify time lag and transport delay.

Functions

- RTCADDONDLL_API LONG [identifyTransportDelay](#) ()
Parameter identification for the transport delay.
- RTCADDONDLL_API double [identifyTimeLag](#) ()
Parameter identification for the time lag.

3.2.1 Detailed Description

Helper functions to identify time lag and transport delay. As the short vector algorithm needs the scan head's time lag and the transport delay of RTC commands, functions to identify these parameters are provided.

Note

- The identification may only be called from an application that has already initialized the RTCxDLL!
- The results are only estimates but give good initial values for the parameters. For optimal results a manual fine tuning is needed. If the marking is shifted along the scan direction TimeLag value has to be decreased. If shifted to the other direction TimeLag value has to be increased.
- A scanner movement is necessary for the identification! Thus a test motion is executed by the routines provided. To ensure the identification process `stop_execution()` is called on entry by the identification routines.

3.2.2 Function Documentation

3.2.2.1 RTCADDONDLL_API double [identifyTimeLag](#) ()

Parameter identification for the time lag.

Returns

Time lag in [us]
-1 in case of an error
-2 in case of RTCx access denied
-3 in case of RTCx timeout error

Examples:

[Example.cpp](#).

3.2.2.2 RTCADDONDLL_API LONG [identifyTransportDelay](#) ()

Parameter identification for the transport delay.

Returns

Transport delay in [us]
-1 in case of an error
-2 in case of RTCx access denied
-3 in case of RTCx timeout error

Examples:

[Example.cpp](#).

3.3 Multi-Board short vector functions

Extension of the [short vectors library](#) to comply with the multi-board commands of the RTCxDLL.

Functions

- RTCADDONDLL_API LONG [n_initShortVectors](#) (UINT CardNo, double TimeLag, UINT TransportDelay, double Speed)
Initializes a Short Vector Handler with dynamic parameters and desired velocity.
- RTCADDONDLL_API LONG [n_initShortVectorsSCANa](#) (UINT CardNo, UINT PreviewTime, UINT Vmax, double Amax, double Speed)
Initializes a Short Vector Handler with dynamic parameters and desired velocity for a scan system with SCANahead control.
- RTCADDONDLL_API LONG [n_setExtraPreRunTime](#) (UINT CardNo, double ExtraPreRunTime)
Sets an additional TimeLag value to increase acceleration distance for first vector of a collinear vector sequence.
- RTCADDONDLL_API LONG [n_setLaserPreTriggerTime](#) (UINT CardNo, double LaserPreTriggerTime)
Sets a pre trigger time for lasers to account for delayed switching.
- RTCADDONDLL_API LONG [n_setLaserSignalShiftSCANa](#) (UINT CardNo, double LaserSignalShift)
Sets a time shift for laser signal switching.
- RTCADDONDLL_API LONG [n_setPreRunWaitTime](#) (UINT CardNo, double PreRunWaitTime)
Sets an additional wait time before each start of a pre-run-sequence.
- RTCADDONDLL_API LONG [n_setMinVectorLengthBits](#) (UINT CardNo, UINT MinVectorLengthBits)
Sets a minimum vector length.
- RTCADDONDLL_API LONG [n_setMinJumpLengthBits](#) (UINT CardNo, UINT MinJumpLengthBits)
Sets a minimum jump length.
- RTCADDONDLL_API LONG [n_addShortVectorCommand](#) (UINT CardNo, double xPos, double yPos, LONG Type)
Adds a new vector command to the preprocessor.
- RTCADDONDLL_API LONG [n_addShortVectorCommand3D](#) (UINT CardNo, double xPos, double yPos, double zPos, LONG Type)
Adds a new 3D vector command to the preprocessor.
- RTCADDONDLL_API LONG [n_finalizeShortVectorInput](#) (UINT CardNo)
*As the algorithm has an internal memory to recognize patterns of collinear vectors there is the possible need to keep the last command internally before adding it to the execution queue.
Calling this function sends a possible last command kept by the handler to the execution queue.
To finalize the sequence the add off an appropriate post-run is included.*
- RTCADDONDLL_API LONG [n_setExecutionListSizeLimit](#) (UINT CardNo, LONG ExeListLimit)
*Sets a warning limit for stored execution list size (as number of entries) to avoid a potential memory exception caused to memory allocation failed.
(This limit value can also be used as a control value for the number of execution list entrys. To estimate the list space requirement on the RTC this value can be approximatly doubled.)*
- RTCADDONDLL_API LONG [n_getNeededListSpace](#) (UINT CardNo)
Reads back the needed list space value of the actual handler.
- RTCADDONDLL_API LONG [n_sendShortVectorCommandsToRtc](#) (UINT CardNo)
Transfers all commands in the command buffer to the RTCx list handled by the application program.
- RTCADDONDLL_API LONG [n_resetShortVectors](#) (UINT CardNo)
Resets the internal state of a specific short vector object.
- RTCADDONDLL_API LONG [n_freeShortVectors](#) (UINT CardNo)
Releases a specific short vector object.

3.3.1 Detailed Description

Extension of the [short vectors library](#) to comply with the multi-board commands of the RTCxDLL.

3.3.2 Function Documentation

3.3.2.1 RTCADDONDLL_API LONG n_addShortVectorCommand (UINT *CardNo*, double *xPos*, double *yPos*, LONG *Type*)

Adds a new vector command to the preprocessor.

Note

For the allowed range of xyz coordinates see the corresponding commands (mark_abs, jump_abs) of the RTC-Manual.

Parameters

in	<i>CardNo</i>	RTCx card number
in	<i>xPos</i>	x coordinate [Bits]
in	<i>yPos</i>	y coordinate [Bits]
in	<i>Type</i>	For JUMP 0, for MARK 1

Returns

- 0 for success
- 1 object handler is invalid
- 2 memory exception
- 3 command type is invalid
- 4 vector length is too short
- 5 warning: execution list size limit exceeded (further description see n_setExecutionListSizeLimit)

3.3.2.2 RTCADDONDLL_API LONG n_addShortVectorCommand3D (UINT *CardNo*, double *xPos*, double *yPos*, double *zPos*, LONG *Type*)

Adds a new 3D vector command to the preprocessor.

Note

For the allowed range of xyz coordinates see the corresponding commands (mark_abs_3d, jump_abs_3d) of the RTC-Manual.

Parameters

in	<i>CardNo</i>	RTCx card number
in	<i>xPos</i>	x coordinate [Bits]
in	<i>yPos</i>	y coordinate [Bits]
in	<i>zPos</i>	z coordinate [Bits]
in	<i>Type</i>	For JUMP 0, for MARK 1

Returns

- 0 for success
- 1 object handler is invalid
- 2 memory exception
- 3 command type is invalid
- 4 vector length is too short
- 5 warning: execution list size limit exceeded (further description see n_setExecutionListSizeLimit)

3.3.2.3 RTCADDONDLL_API LONG n_finalizeShortVectorInput (UINT *CardNo*)

As the algorithm has an internal memory to recognize patterns of collinear vectors there is the possible need to keep the last command internally before adding it to the execution queue.

Calling this function sends a possible last command kept by the handler to the execution queue.

To finalize the sequence the add off an appropriate post-run is included.

Parameters

in	<i>CardNo</i>	RTCx card number related to the short vector handler to be finalized
----	---------------	--

Returns

- 0 for success
- 1 object handler is invalid

3.3.2.4 RTCADDONDLL_API LONG n_freeShortVectors (UINT *CardNo*)

Releases a specific short vector object.

Parameters

in	<i>CardNo</i>	RTCx card number of which short vector handler to be released
----	---------------	---

Returns

- 0 for success
- 1 object handler is invalid

3.3.2.5 RTCADDONDLL_API LONG n_getNeededListSpace (UINT *CardNo*)

Reads back the needed list space value of the actual handler.

Parameters

in	<i>CardNo</i>	card number of RTCx
----	---------------	---------------------

Returns

- 0 in case of execution list is empty
- ListSpace* in case of execution list is not empty
- 1 object handler invalid
- 2 memory exception

3.3.2.6 RTCADDONDLL_API LONG n_initShortVectors (UINT *CardNo*, double *TimeLag*, UINT *TransportDelay*, double *Speed*)

Initializes a Short Vector Handler with dynamic parameters and desired velocity.

The function will construct a new short vectors object for a specific RTCx card in the PC.

Parameters

in	<i>CardNo</i>	RTCx card number
in	<i>TimeLag</i>	Controller time lag (depends on scan head tuning) [us]
in	<i>TransportDelay</i>	Time needed to transfer RTCx commands to the scan head [us]
in	<i>Speed</i>	[Bits/ms]

Note

Time lag will be checked to be at least 20 us!

Returns

- 0 for success
- 1 if memory allocation failed
- 2 for bad input
- 3 ShortVectors functions access denied
- 4 version mismatch of RTCxAddOnDLL version and RTCx DLL, HEX (OUT-File) or RTC (RBF-File) version
- 5 CardNo has to be greater than zero

3.3.2.7 RTCADDONDLL_API LONG n_initShortVectorsSCANa (UINT *CardNo*, UINT *PreviewTime*, UINT *Vmax*, double *Amax*, double *Speed*)

Initializes a Short Vector Handler with dynamic parameters and desired velocity for a scan system with SCANahead control.

The function will construct a new short vectors object for a specific RTC6 card in the PC.

Note

This function only works with RTC6 and SCANa option enabled. In addition it is mandatory to activate the RTC6-SCANahead functionality via set_scanahead_params at the latest before starting the list execution.

Parameters

in	<i>CardNo</i>	RTC6 card number
in	<i>PreviewTime</i>	Pre-calculation time for galvanometer scanner control in [10 us]
in	<i>Vmax</i>	Velocity limit as control bits (galvanometer scanner rotation angle) [bits / 10 us]
in	<i>Amax</i>	Acceleration limit as control bits (galvanometer scanner rotation angle) [bits / (10 us) ²]
in	<i>Speed</i>	[Bits/ms]

Note

Time lag will be checked to be at least 20 us!

Returns

- 0 for success
- 1 if memory allocation failed
- 2 for bad input
- 3 ShortVectors functions access denied
- 4 version mismatch of RTCxAddOnDLL version and RTCx DLL, HEX (OUT-File) or RTC (RBF-File) version
- 5 CardNo has to be greater than zero
- 6 Speed value exceeds Vmax

3.3.2.8 RTCADDONDLL_API LONG n_resetShortVectors (UINT *CardNo*)

Resets the internal state of a specific short vector object.

The command buffer and a possible last command held internally are cleared.

Parameters

in	<i>CardNo</i>	RTCx card number of which short vector handler to be reset
----	---------------	--

Returns

- 0 for success
- 1 object handler is invalid

3.3.2.9 RTCADDONDLL_API LONG n_sendShortVectorCommandsToRtc (UINT *CardNo*)

Transfers all commands in the command buffer to the RTCx list handled by the application program.

Note

This function may only be called from an application which has initialized the RTCxDLL and started an RTC list before calling!!

Parameters

in	<i>CardNo</i>	card number of RTCx
----	---------------	---------------------

Returns

Number of list commands written to the RTC

- 1 in case of a failure
- 2 in case list memory is full

3.3.2.10 RTCADDONDLL_API LONG n_setExecutionListSizeLimit (UINT *CardNo*, LONG *ExeListLimit*)

Sets a warning limit for stored execution list size (as number of entries) to avoid a potential memory exception caused to memory allocation failed.

(This limit value can also be used as a control value for the number of execution list entries. To estimate the list space requirement on the RTC this value can be approximatly doubled.)

Note

- If the limit is exceeded addShortVectorCommand returns with ReturnCode = -5.
This return code is not to be understood as an error code, but only as a warning and has no effect on the execution of the addShortVectorCommand function.
It advises you to call the sendShortVectorCommandsToRtc command as soon as possible to empty the handler's cached execution list to free memory.
- One entry of the execution list needs about 440 byte RAM.
Recommended value for parameter ExeListLimit:
 $\text{ExeListLimit} = \text{available memory [byte]} / 440 \text{ byte}$
- The limit check is deactivated by default initialization.
It needs to be explicitly activated by calling the function with parameter $\text{ExeListLimit} > 0$.
For deactivation call the function with parameter $\text{ExeListLimit} \leq 0$.

Parameters

in	<i>CardNo</i>	card number of RTCx
in	<i>ExeListLimit</i>	warning limit for handler execution list size

Returns

- 0 in case of success
- 1 object handler invalid
- 2 memory exception

3.3.2.11 RTCADDONDLL_API LONG n_setExtraPreRunTime (UINT *CardNo*, double *ExtraPreRunTime*)

Sets an additional TimeLag value to increase acceleration distance for first vector of a collinear vector sequence.

Note

If the first vector / vectors of a collinear dash line are too short or too long the value of ExtraPreRunTime needs to be increased.

Parameters

in	<i>CardNo</i>	RTCx card number
in	<i>ExtraPreRunTime</i>	TimeLag value to extend acceleration distance [us]

Returns

- 0 for success
- 1 object handler or command type is invalid
- 2 memory exception

3.3.2.12 RTCADDONDLL_API LONG n_setLaserPreTriggerTime (UINT *CardNo*, double *LaserPreTriggerTime*)

Sets a pre trigger time for lasers to account for delayed switching.

The laser on signal will be set as specified before the marking vector starts.

Parameters

in	<i>CardNo</i>	RTCx card number
in	<i>LaserPreTriggerTime</i>	Time to compensate for delayed switching (≥ 0)

Returns

- 0 for success
- 1 object handler or command type is invalid
- 2 memory exception

3.3.2.13 RTCADDONDLL_API LONG n_setLaserSignalShiftSCANa (UINT *CardNo*, double *LaserSignalShift*)

Sets a time shift for laser signal switching.

The laser signals will be shifted by the specified time shift.

Note

This function only works with RTC6 and SCANa option enabled.
If a time shift shall be executed, it needs to be called once before the first call of sendShortVectorCommandsToRtc.
Please remark, the value must not be changed within a RTC list.

Parameters

in	<i>CardNo</i>	RTCx card number
in	<i>LaserSignalShift</i>	Time shift [us] to synchronize laser switching and scanner movement (allowed range -20us to +20us)

Returns

- 0 for success
- 1 object handler or command type is invalid
- 2 memory exception

- 3 for LaserSignalShift value is out of range and has been clipped
- 4 for SCANa option is not enabled

3.3.2.14 RTCADDONDLL_API LONG n_setMinJumpLengthBits (UINT *CardNo*, UINT *MinJumpLengthBits*)

Sets a minimum jump length.

Only jump vectors of at least the specified minimum length will be added to the preprocessor.

Note

The minimum length must be at least equivalent to a jump duration of 0.5 us (RTC6) / 10 us (RTC5). Shorter jumps cannot be processed properly.

Parameters

in	<i>CardNo</i>	RTCx card number
in	<i>MinJumpLengthBits</i>	Minimum jump length in Bits

Returns

- 0 for success
- 1 object handler invalid or MinJumpLengthBits too small
- 2 memory exception

3.3.2.15 RTCADDONDLL_API LONG n_setMinVectorLengthBits (UINT *CardNo*, UINT *MinVectorLengthBits*)

Sets a minimum vector length.

Only mark vectors of at least the specified minimum length will be added to the preprocessor.

Note

The minimum length must be at least equivalent to a vector duration of 0.5 us. Shorter vectors cannot be processed properly.

Parameters

in	<i>CardNo</i>	RTCx card number
in	<i>MinVectorLengthBits</i>	Minimum vector length in Bits

Returns

- 0 for success
- 1 object handler invalid or MinVectorLengthBits too small
- 2 memory exception

3.3.2.16 RTCADDONDLL_API LONG n_setPreRunWaitTime (UINT *CardNo*, double *PreRunWaitTime*)

Sets an additional wait time before each start of a pre-run-sequence.

Note

It is allowed to change the value of parameter PreRunWaitTime while adding ShortVectorCommands to the ShortVectorHandler.

The set value for PreRunWaitTime applies to the current call of addShortVectorCommand and all subsequent calls until the value is changed by calling [n_setPreRunWaitTime\(\)](#) with another parameter value.

By default initialization PreRunWaitTime is set to 0.

Parameters

<i>in</i>	<i>CardNo</i>	RTCx card number
<i>in</i>	<i>PreRunWaitTime</i>	Wait time [us] between post-run und pre-run of two non-collinear vectors (allowed range 0 us to 1,000,000 us) The parameter is rounded to an integer-multiple of 10

Returns

- 0 for success
- 1 object handler or command type is invalid
- 2 memory exception
- 3 for PreRunWaitTime value is out of range and has been clipped

3.4 Multi-Board identification routines

Extension of the [identification functions](#) to comply with the multi-board commands of the RTCxDLL.

Functions

- RTCADDONDLL_API LONG [n_identifyTransportDelay](#) (UINT CardNo)
Parameter identification for the transport delay.
- RTCADDONDLL_API double [n_identifyTimeLag](#) (UINT CardNo)
Parameter identification for the time lag.

3.4.1 Detailed Description

Extension of the [identification functions](#) to comply with the multi-board commands of the RTCxDLL.

3.4.2 Function Documentation

3.4.2.1 RTCADDONDLL_API double [n_identifyTimeLag](#) (UINT *CardNo*)

Parameter identification for the time lag.

Parameters

in	<i>CardNo</i>	Number of used RTCx card
----	---------------	--------------------------

Returns

Time lag in [us]
-1 in case of an error
-2 in case of RTCx access denied
-3 in case of RTCx timeout error

3.4.2.2 RTCADDONDLL_API LONG [n_identifyTransportDelay](#) (UINT *CardNo*)

Parameter identification for the transport delay.

Parameters

in	<i>CardNo</i>	Number of used RTCx card
----	---------------	--------------------------

Returns

Transport delay in [us]
-1 in case of an error
-2 in case of RTCx access denied
-3 in case of RTCx timeout error

4 Example Documentation

4.1 Example.cpp

Author

SCANLAB GmbH

Date

December 2018

```
// insert define if RTC5 is to be used; comment define if RTC6 is to be used
#define RTC5_MODE

#if defined RTC5_MODE // use RTC5
#include "RTC5ADDONimpl.h"
#include "RTC5impl.h"
#else
#include "RTC6ADDONimpl.h"
#include "RTC6impl.h"
#define SCANA // insert define SCANA if a scan head with SCANAhead functionality (excelliSCAN)
// is connected; comment define if an intelliSCAN is connected
#endif

// function provided by the user!
// returns xPos and yPos in RTC Bit; Command Type 0 for jumps and 1 for marks
extern bool getNextTarget(double *xPos, double *yPos, int *Command);
// ...

// declaration of example transfer functions of short vector commands.
int markShortVectors(int TransportDelay, double TimeLag, double ExtraPreRunTime, double LaserPreTriggerTime
, double Speed, unsigned int PreviewTime, unsigned int Vmax, double Amax, double LaserSignalShift);
int markShortVectorsContinuousSending(int TransportDelay, double TimeLag, double ExtraPreRunTime, double
LaserPreTriggerTime, double Speed, unsigned int PreviewTime, unsigned int Vmax, double Amax, double
LaserSignalShift);

int rtcShortVectorList()
{
    // scan system parameters
    // parameters for intelliSCAN
    double TimeLag = 0.0;
    int TransportDelay = 0;
    double ExtraPreRunTime = 0.0;
    // parameters for excelliSCAN (SCANA)
    unsigned int PreviewTime = 0;
    unsigned int Vmax = 0;
    double Amax = 0.0;
    unsigned int Mode = 1; // read parameters form scan head
    unsigned int HeadNo = 1; // connected to primary scan head connector
    unsigned int TableNo = 1; // chose used correction file table number 1-4
    double LaserSignalShift = 0.0; // Laser dependent [us]; allowed range [-20 ... 20]

    // additional parameters for laser switching
    double LaserPreTriggerTime = 0.0; // Laser dependent [us]

    // scan parameters
    double Speed = 3210; // Your marking speed [Bits/ms]

    // initialize as RTC as usual
    // ... insert your code ...

    // activate ShortVectors functions
    ULONG_PTR ActivationCode = NULL;
    activateShortVectors( ActivationCode );

    // run identification routines - ensure scan head is connected to RTC and power supply
    #if defined(SCANA)
    unsigned int ErrorCode = 0;
    ErrorCode = get_scanahead_params(HeadNo, &PreviewTime, &Vmax, &Amax);
    // ... insert error handling ...
    // if not formerly executed - execute set_scanahead_params to activate RTC6-SCANAhead-functionality
    ErrorCode = set_scanahead_params(Mode, HeadNo, TableNo, PreviewTime, Vmax, Amax);
    // ... insert error handling ...
    #else // run identification routines for intelliScan
    // run the identification
    TimeLag = identifyTimeLag();
    TransportDelay = identifyTransportDelay();
    #endif
}
```

```

    if (TimeLag < 0 || TransportDelay < 0 )
    {
        // Error!
    }
#endif

    // list handling...
    set_start_list(1);
    set_jump_speed(Speed); // jump speed and mark speed have to be defined to the same value
    set_mark_speed(Speed);
    set_scanner_delays(0,0,0); // deactivate all scanner delays (scanahead auto delays are
                               // automatically deactivated by Short-Vector routine)

    // Insert the short vector commands into the list
    markShortVectors(TransportDelay, TimeLag, ExtraPreRunTime, LaserPreTriggerTime, Speed, PreviewTime,
                    Vmax, Amax, LaserSignalShift);
    // alternatively look into:
    // markShortVectorsContinuousSending(TransportDelay, TimeLag, ExtraPreRunTime, Speed, PreviewTime,
    //                                   Vmax, Amax, LaserSignalShift);

    // list execution
    set_end_of_list();
    execute_list(1);
}

// This transfers short vector commands as a block after completing the input.
int markShortVectors(int TransportDelay, double TimeLag, double ExtraPreRunTime, double LaserPreTriggerTime
                    , double Speed, unsigned int PreviewTime, unsigned int Vmax, double Amax, double LaserSignalShift)
{
    int ListCommandCounter = 0;
    double xTarget, yTarget;
    int CommandType;

    LONG ShortVectorHandler = 0;
    #if defined(SCANa)
        initShortVectorsSCANa(&ShortVectorHandler, PreviewTime, Vmax, Amax, Speed);
        setLaserSignalShiftSCANa(ShortVectorHandler, LaserSignalShift);
    #else
        initShortVectors(&ShortVectorHandler, TimeLag, TransportDelay, Speed);
        setExtraPreRunTime(ShortVectorHandler, ExtraPreRunTime);
    #endif
    setLaserPreTriggerTime(ShortVectorHandler, LaserPreTriggerTime);

    // read next command from your input!!
    while (getNextTarget(&xTarget, &yTarget, &CommandType))
    {
        addShortVectorCommand(ShortVectorHandler, xTarget, yTarget, CommandType);
    }

    finalizeShortVectorInput(ShortVectorHandler);
    ListCommandCounter = sendShortVectorCommandsToRtc(ShortVectorHandler);

    freeShortVectors(ShortVectorHandler);
    return ListCommandCounter;
}

// This transfers short vector commands immediately after each calculation
int markShortVectorsContinuousSending(int TransportDelay, double TimeLag, double ExtraPreRunTime, double
    LaserPreTriggerTime, double Speed, unsigned int PreviewTime, unsigned int Vmax, double Amax, double
    LaserSignalShift)
{
    int ListCommandCounter = 0;
    double xTarget, yTarget;
    int CommandType;

    LONG ShortVectorHandler = 0;

    #if defined(SCANa)
        initShortVectorsSCANa(&ShortVectorHandler, PreviewTime, Vmax, Amax, Speed);
        setLaserSignalShiftSCANa(ShortVectorHandler, LaserSignalShift);
    #else
        initShortVectors(&ShortVectorHandler, TimeLag, TransportDelay, Speed);
        setExtraPreRunTime(ShortVectorHandler, ExtraPreRunTime);
    #endif
    setLaserPreTriggerTime(ShortVectorHandler, LaserPreTriggerTime);

    // read next command from your input!!
    while (getNextTarget(&xTarget, &yTarget, &CommandType))
    {
        addShortVectorCommand(ShortVectorHandler, xTarget, yTarget, CommandType);
        ListCommandCounter += sendShortVectorCommandsToRtc(ShortVectorHandler);
    }
    finalizeShortVectorInput(ShortVectorHandler);
}

```

```
ListCommandCounter += sendShortVectorCommandsToRtc(ShortVectorHandler);  
freeShortVectors(ShortVectorHandler);  
return ListCommandCounter;  
}
```

Index

- activateShortVectors
 - Short vector optimization, [5](#)
- addShortVectorCommand
 - Short vector optimization, [5](#)
- addShortVectorCommand3D
 - Short vector optimization, [6](#)
- finalizeShortVectorInput
 - Short vector optimization, [6](#)
- freeAllShortVectors
 - Short vector optimization, [6](#)
- freeShortVectors
 - Short vector optimization, [6](#)
- getNeededListSpace
 - Short vector optimization, [7](#)
- Identification Routines, [13](#)
 - identifyTimeLag, [13](#)
 - identifyTransportDelay, [13](#)
- identifyTimeLag
 - Identification Routines, [13](#)
- identifyTransportDelay
 - Identification Routines, [13](#)
- initShortVectors
 - Short vector optimization, [7](#)
- initShortVectorsSCANa
 - Short vector optimization, [8](#)
- Multi-Board identification routines, [22](#)
 - n_identifyTimeLag, [22](#)
 - n_identifyTransportDelay, [22](#)
- Multi-Board short vector functions, [14](#)
 - n_addShortVectorCommand, [15](#)
 - n_addShortVectorCommand3D, [15](#)
 - n_finalizeShortVectorInput, [15](#)
 - n_freeShortVectors, [16](#)
 - n_getNeededListSpace, [16](#)
 - n_initShortVectors, [16](#)
 - n_initShortVectorsSCANa, [17](#)
 - n_resetShortVectors, [17](#)
 - n_sendShortVectorCommandsToRtc, [17](#)
 - n_setExecutionListSizeLimit, [18](#)
 - n_setExtraPreRunTime, [18](#)
 - n_setLaserPreTriggerTime, [19](#)
 - n_setLaserSignalShiftSCANa, [19](#)
 - n_setMinJumpLengthBits, [20](#)
 - n_setMinVectorLengthBits, [20](#)
 - n_setPreRunWaitTime, [20](#)
- n_addShortVectorCommand
 - Multi-Board short vector functions, [15](#)
- n_addShortVectorCommand3D
 - Multi-Board short vector functions, [15](#)
- n_finalizeShortVectorInput
 - Multi-Board short vector functions, [15](#)
- n_freeShortVectors
 - Multi-Board short vector functions, [16](#)
- n_getNeededListSpace
 - Multi-Board short vector functions, [16](#)
- n_identifyTimeLag
 - Multi-Board identification routines, [22](#)
- n_identifyTransportDelay
 - Multi-Board identification routines, [22](#)
- n_initShortVectors
 - Multi-Board short vector functions, [16](#)
- n_initShortVectorsSCANa
 - Multi-Board short vector functions, [17](#)
- n_resetShortVectors
 - Multi-Board short vector functions, [17](#)
- n_sendShortVectorCommandsToRtc
 - Multi-Board short vector functions, [17](#)
- n_setExecutionListSizeLimit
 - Multi-Board short vector functions, [18](#)
- n_setExtraPreRunTime
 - Multi-Board short vector functions, [18](#)
- n_setLaserPreTriggerTime
 - Multi-Board short vector functions, [19](#)
- n_setLaserSignalShiftSCANa
 - Multi-Board short vector functions, [19](#)
- n_setMinJumpLengthBits
 - Multi-Board short vector functions, [20](#)
- n_setMinVectorLengthBits
 - Multi-Board short vector functions, [20](#)
- n_setPreRunWaitTime
 - Multi-Board short vector functions, [20](#)
- resetShortVectors
 - Short vector optimization, [8](#)
- sendShortVectorCommandsToRtc
 - Short vector optimization, [9](#)
- setExecutionListSizeLimit
 - Short vector optimization, [9](#)
- setExtraPreRunTime
 - Short vector optimization, [10](#)
- setLaserPreTriggerTime
 - Short vector optimization, [10](#)
- setLaserSignalShiftSCANa
 - Short vector optimization, [10](#)
- setMinJumpLengthBits
 - Short vector optimization, [11](#)
- setMinVectorLengthBits
 - Short vector optimization, [11](#)
- setPreRunWaitTime
 - Short vector optimization, [12](#)
- Short vector optimization, [3](#)
 - activateShortVectors, [5](#)
 - addShortVectorCommand, [5](#)
 - addShortVectorCommand3D, [6](#)
 - finalizeShortVectorInput, [6](#)
 - freeAllShortVectors, [6](#)

freeShortVectors, [6](#)
getNeededListSpace, [7](#)
initShortVectors, [7](#)
initShortVectorsSCANa, [8](#)
resetShortVectors, [8](#)
sendShortVectorCommandsToRtc, [9](#)
setExecutionListSizeLimit, [9](#)
setExtraPreRunTime, [10](#)
setLaserPreTriggerTime, [10](#)
setLaserSignalShiftSCANa, [10](#)
setMinJumpLengthBits, [11](#)
setMinVectorLengthBits, [11](#)
setPreRunWaitTime, [12](#)